ד״ר ולד לנדא

Vibe Coding

שיעור 3 - Prompt Engineering

מטרות המפגש:

1. להסביר למה ניסוח הפרומפט משנה את איכות הקוד - ולזהות "החלטות פתוחות" בפרומפט.
2. לכתוב פרומפט שכולל את ארבע אבני היסוד: **הקשר**, **מטרה**, **אילוצים**, **פורמט** - ולהשתמש בתבנית פרומפט לשימוש חוזר.
3. לבנות פרומפט ארוך עם מבנה ותוחמים, כך שהמודל מבחין בין הוראות לבין חומר (קוד, נתונים, שגיאות).
4. לשפר פרומפט באיטרציה, כולל החזרת הודעת שגיאה מלאה למודל בצורה שמביאה לתיקון אמיתי.
5. להשתמש ב-**few-shot** - ללמד את המודל דפוס דרך דוגמאות קלט→פלט, ולהשתמש באותן דוגמאות כבדיקה.
6. לפרק משימה גדולה לצעדים קטנים שכל אחד מהם נבדק בנפרד.
7. ליישם שלוש טכניקות משלימות: **שאלות הבהרה**, **תוכנית לפני קוד**, **ותפקיד** (כולל ההבחנה בין הנחיית מערכת להודעת משתמש) - ולדעת מה כל אחת **לא** עושה.
8. לנסח פרומפט כהתניה $P_\theta(y \mid x)$, להגדיר את איכותו כהסתברות ההצלחה $P_{\text{succ}}(x)$ שהפלט עומד במפרט, ולנתח בעזרת מודלים הסתברותיים פשוטים (מכפלה, התפלגות גיאומטרית, אמינות שרשרת) למה כל אחת מהטכניקות עובדת.
9. להשוות בין שני פרומפטים: לאמוד הסתברות הצלחה עם רווח סמך, ולפרש אם ההבדל ביניהם מובהק.

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: "*כתוב קוד לבדיקת מספר ראשוני*"

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: "*כתוב קוד לבדיקת מספר ראשוני*"

```python
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: "*כתוב קוד לבדיקת מספר ראשוני*"

```python
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

- מה קורה כאשר בודקים?:

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: "*כתוב קוד לבדיקת מספר ראשוני*"

```python
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

- מה קורה כאשר בודקים?:

is_prime(97) → True ✓
is_prime(9) → False ✓
is_prime(2) → True ✓
is_prime(1) → True ✗ (אינו ראשוני 1)
is_prime(0) → True ✗
is_prime(-7) → True ✗

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: "*כתוב קוד לבדיקת מספר ראשוני*"

```python
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

- מה קורה כאשר בודקים?:

is_prime(97) → True ✓
is_prime(9) → False ✓
is_prime(2) → True ✓
is_prime(1) → True ✗ (אינו ראשוני 1)
is_prime(0) → True ✗
is_prime(-7) → True ✗

- **האם יש עוד בעיה?**

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: "*כתוב קוד לבדיקת מספר ראשוני*"

```python
def is_prime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

- מה קורה כאשר בודקים?:

is_prime(97) → True ✓
is_prime(9) → False ✓
is_prime(2) → True ✓
is_prime(1) → True ✗ (אינו ראשוני 1)
is_prime(0) → True ✗
is_prime(-7) → True ✗

- **האם יש עוד בעיה?** הלולאה רצה עד *n*, אין התיחסות ליעילות!

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: *"כתוב פונקציה ב-Python בשם is_prime(n) שמקבלת מספר שלם ומחזירה True אם הוא ראשוני ו-False אחרת.*
  - מספרים קטנים או שווים ל-1 (כולל 0 ושליליים) אינם ראשוניים.
  - הפונקציה צריכה להיות יעילה גם למספרים בסדר גודל של מיליארד (בדיקה עד שורש n).
  - בלי ספריות חיצוניות.
  החזר: את הפונקציה עם type hints ו-docstring, ומתחתיה הסבר קצר בעברית."

ד"ר ולד לנדא

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: *"כתוב פונקציה ב-Python בשם is_prime(n) שמקבלת מספר שלם ומחזירה True אם הוא ראשוני ו-False אחרת.*
  - מספרים קטנים או שווים ל-1 (כולל 0 ושליליים) אינם ראשוניים.
  - הפונקציה צריכה להיות יעילה גם למספרים בסדר גודל של מיליארד (בדיקה עד שורש n).
  - בלי ספריות חיצוניות.
  החזר: את הפונקציה עם type hints ו-docstring, ומתחתיה הסבר קצר בעברית."

```python
import math

def is_prime(n: int) -> bool:
    """Return True if n is a prime number, False otherwise."""
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False
    for d in range(3, math.isqrt(n) + 1, 2):
        if n % d == 0:
            return False
    return True
```

ד"ר ולד לנדא

Vibe Coding

שיעור 3 - חלק א' - המחשה: אותה מטרה, שני פרומפטים

- שני פרומפטים לאותה מטרה, לאותו מודל, יכולים להחזיר קוד שאחד מהם שגוי והשני נכון.
- פרומפט: *"כתוב פונקציה ב-Python בשם is_prime(n) שמקבלת מספר שלם ומחזירה True אם הוא ראשוני ו-False אחרת.*
  - מספרים קטנים או שווים ל-1 (כולל 0 ושליליים) אינם ראשוניים.
  - הפונקציה צריכה להיות יעילה גם למספרים בסדר גודל של מיליארד (בדיקה עד שורש n).
  - בלי ספריות חיצוניות.
  החזר: את הפונקציה עם type hints ו-docstring, ומתחתיה הסבר קצר בעברית."

```python
import math

def is_prime(n: int) -> bool:
    """Return True if n is a prime number, False otherwise."""
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False
    for d in range(3, math.isqrt(n) + 1, 2):
        if n % d == 0:
            return False
    return True
```

- **קוד שרץ ≠ קוד נכון.**

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

- **ספירת "החלטות פתוחות" -** הוא הכלי לשפר פרומפט בצורה שיטתית ע״י מציאת גורמים החסרים בו.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

- **ספירת "החלטות פתוחות" -** הוא הכלי לשפר פרומפט בצורה שיטתית ע״י מציאת גורמים החסרים בו.
- כל משימת תכנות כוללת עשרות **החלטות** קטנות: **שפה**, **שם פונקציה**, **סוג קלט**, **סוג פלט**, **מה עושים בקלט לא תקין**, **יעילות**, **ספריות**, **סגנון**, **ואיך להחזיר את התשובה**.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

- **ספירת "החלטות פתוחות" -** הוא הכלי לשפר פרומפט בצורה שיטתית ע״י מציאת גורמים החסרים בו.
- כל משימת תכנות כוללת עשרות **החלטות** קטנות: **שפה**, **שם פונקציה**, **סוג קלט**, **סוג פלט**, **מה עושים בקלט לא תקין**, **יעילות**, **ספריות**, **סגנון**, **ואיך להחזיר את התשובה**.
- כל החלטה שלא נכתבתה בפרומפט היא **החלטה פתוחה** - והמודל ממלא אותה **בברירת המחדל הסבירה ביותר** מנתוני האימון שלו. לפעמים ברירת המחדל מתאימה לכם, ולעיתים קרובות לא.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

- **ספירת "החלטות פתוחות" -** הוא הכלי לשפר פרומפט בצורה שיטתית ע״י מציאת גורמים החסרים בו.
- כל משימת תכנות כוללת עשרות **החלטות** קטנות: **שפה**, **שם פונקציה**, **סוג קלט**, **סוג פלט**, **מה עושים בקלט לא תקין**, **יעילות**, **ספריות**, **סגנון**, **ואיך להחזיר את התשובה**.
- כל החלטה שלא נכתבתה בפרומפט היא **החלטה פתוחה** - והמודל ממלא אותה **בברירת המחדל הסבירה ביותר** מנתוני האימון שלו. לפעמים ברירת המחדל מתאימה לכם, ולעיתים קרובות לא.

- מכירים את הביטוי?: **Garbage in → garbage out**. קלט מעורפל = פלט מעורפל.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

- **ספירת "החלטות פתוחות" -** הוא הכלי לשפר פרומפט בצורה שיטתית ע״י מציאת גורמים החסרים בו.
- כל משימת תכנות כוללת עשרות **החלטות** קטנות: **שפה**, **שם פונקציה**, **סוג קלט**, **סוג פלט**, **מה עושים בקלט לא תקין**, **יעילות**, **ספריות**, **סגנון**, **ואיך להחזיר את התשובה**.
- כל החלטה שלא נכתבתה בפרומפט היא **החלטה פתוחה** - והמודל ממלא אותה **בברירת המחדל הסבירה ביותר** מנתוני האימון שלו. לפעמים ברירת המחדל מתאימה לכם, ולעיתים קרובות לא.

- מכירים את הביטוי?: **Garbage in → garbage out**. קלט מעורפל = פלט מעורפל.

חזרה לדוגמא:

הפרומפט `"כתוב קוד לבדיקת מספר ראשוני"`:

החלטה	נקבעה בפרומפט?	מה המודל עשה בפועל
שפה	לא	בחר Python (הנפוצה ביותר)
שם וחתימה	לא	`is_prime(n)`
מה מחזירים	לא	`True/False`
מקרי קצה (0, 1, שליליים)	לא	לא טיפל — שגוי
יעילות	לא	לולאה עד n — איטי מאוד במספרים גדולים
קלט מהמשתמש או פונקציה?	לא	פונקציה (לפעמים מודלים מוסיפים `input()`)
הסבר	לא	ללא

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

- **ספירת "החלטות פתוחות" -** הוא הכלי לשפר פרומפט בצורה שיטתית ע״י מציאת גורמים החסרים בו.
- כל משימת תכנות כוללת עשרות **החלטות** קטנות: **שפה**, **שם פונקציה**, **סוג קלט**, **סוג פלט**, **מה עושים בקלט לא תקין**, **יעילות**, **ספריות**, **סגנון**, **ואיך להחזיר את התשובה**.
- כל החלטה שלא נכתבתה בפרומפט היא **החלטה פתוחה** - והמודל ממלא אותה **בברירת המחדל הסבירה ביותר** מנתוני האימון שלו. לפעמים ברירת המחדל מתאימה לכם, ולעיתים קרובות לא.

- מכירים את הביטוי?: **Garbage in → garbage out**. קלט מעורפל = פלט מעורפל.

חזרה לדוגמא:

הפרומפט `"כתוב קוד לבדיקת מספר ראשוני"`:

מה המודל עשה בפועל	נקבעה בפרומפט?	החלטה
בחר Python (הנפוצה ביותר)	לא	שפה
`is_prime(n)`	לא	שם וחתימה
`True/False`	לא	מה מחזירים
לא טיפל — שגוי	לא	מקרי קצה (0, 1, שליליים)
לולאה עד n — איטי מאוד במספרים גדולים	לא	יעילות
פונקציה (לפעמים מודלים מוסיפים `input()`)	לא	קלט מהמשתמש או פונקציה?
ללא	לא	הסבר

- **7 החלטות פתוחות בפרומפט של 5 מילים.**
  הפרומפט החזק מחלק א' סגר את כולן.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>המודל כהתפלגות מותנית:</u>

- **הסתברות ההצלחה של פרומפט :** נסמן ב- S את **קבוצת התשובות התקינות**: תשובות שהקוד שבהן עומד במפרט שלכם (במקרה של `*is_prime*`: הפונקציה מחזירה את הערך הנכון לכל מספר שלם, בזמן סביר).

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>המודל כהתפלגות מותנית:</u>

- **הסתברות ההצלחה של פרומפט :** נסמן ב- S את **קבוצת התשובות התקינות**: תשובות שהקוד שבהן עומד במפרט שלכם (במקרה של `is_prime`: הפונקציה מחזירה את הערך הנכון לכל מספר שלם, בזמן סביר).
- **איכות הפרומפט** היא **ההסתברות שתשובה** שהמודל ייצר תהיה **בתוך** S :

$$P_{\text{succ}}(x) = \Pr_{y \sim P_\theta(\cdot|x)} \left[y \in S \right]$$

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>המודל כהתפלגות מותנית:</u>

- **הסתברות ההצלחה של פרומפט :** נסמן ב- S את **קבוצת התשובות התקינות**: תשובות שהקוד שבהן עומד במפרט שלכם (במקרה של `*is_prime*`: הפונקציה מחזירה את הערך הנכון לכל מספר שלם, בזמן סביר).
- **איכות הפרומפט** היא **ההסתברות שתשובה** שהמודל ייצר תהיה **בתוך** S :

$$P_{\text{succ}}(x) = \Pr_{y \sim P_\theta(\cdot|x)} \left[y \in S \right]$$

- הסימון $y \sim P_\theta(\cdot \mid x)$ פירושו "y נדגמת מההתפלגות של המודל, בהינתן x".

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>המודל כהתפלגות מותנית:</u>

- **הסתברות ההצלחה של פרומפט :** נסמן ב- S את **קבוצת התשובות התקינות**: תשובות שהקוד שבהן עומד במפרט שלכם (במקרה של `is_prime`: הפונקציה מחזירה את הערך הנכון לכל מספר שלם, בזמן סביר).
- **איכות הפרומפט** היא **ההסתברות שתשובה** שהמודל ייצר תהיה **בתוך** S :

$$P_{\text{succ}}(x) = \Pr_{y \sim P_\theta(\cdot|x)} \left[y \in S \right]$$

- הסימון $y \sim P_\theta(\cdot \mid x)$ פירושו "y נדגמת מההתפלגות של המודל, בהינתן x".

מסכנות:

1. **איכות היא תכונה של הפרומפט, לא של תשובה בודדת.**

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>המודל כהתפלגות מותנית:</u>

- **הסתברות ההצלחה של פרומפט :** נסמן ב- S את **קבוצת התשובות התקינות**: תשובות שהקוד שבהן עומד במפרט שלכם (במקרה של `*is_prime*`: הפונקציה מחזירה את הערך הנכון לכל מספר שלם, בזמן סביר).
- **איכות הפרומפט** היא **ההסתברות שתשובה** שהמודל ייצר תהיה **בתוך** S :

$$P_{\mathrm{succ}}(x) = \Pr_{y \sim P_\theta(\cdot|x)} \left[y \in S \right]$$

- הסימון $y \sim P_\theta(\cdot \mid x)$ פירושו "y נדגמת מההתפלגות של המודל, בהינתן x".

מסכנות:

1. **איכות היא תכונה של הפרומפט, לא של תשובה בודדת.**
2. **איכות תלויה גם במודל ובמפרט** - $P_{\mathrm{succ}}(x)$ מחושבת עבור מודל מסוים (ו-θ שלו).

ד"ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>המודל כהתפלגות מותנית:</u>

- **הסתברות ההצלחה של פרומפט :** נסמן ב- S את **קבוצת התשובות התקינות**: תשובות שהקוד שבהן עומד במפרט שלכם (במקרה של `is_prime`: הפונקציה מחזירה את הערך הנכון לכל מספר שלם, בזמן סביר).
- **איכות הפרומפט** היא **ההסתברות שתשובה** שהמודל ייצר תהיה **בתוך** S :

$$P_{\text{succ}}(x) = \Pr_{y \sim P_\theta(\cdot|x)} \left[y \in S \right]$$

- הסימון $y \sim P_\theta(\cdot \mid x)$ פירושו "y נדגמת מההתפלגות של המודל, בהינתן x".

מסכנות:

1. **איכות היא תכונה של הפרומפט, לא של תשובה בודדת.**
2. **איכות תלויה גם במודל ובמפרט** - $P_{\text{succ}}(x)$ מחושבת עבור מודל מסוים (ו-θ שלו).
3. **בפועל מודדים "עבר את הבדיקות", לא האם "נכון"** - ההסתברות "לעבור את הבדיקות" **גדולה או שווה** ל- $P_{\text{succ}}(x)$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

נניח שהמשימה כוללת D החלטות, ונסמן אותן $\ell = 1, \dots, D$. נסמן ב-g_ℓ את ההסתברות שהתשובה מתאימה למפרט שלכם בהחלטה ℓ:

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

נניח שהמשימה כוללת D החלטות, ונסמן אותן $\ell = 1, \dots, D$. נסמן ב-g_ℓ את ההסתברות שהתשובה מתאימה למפרט שלכם בהחלטה ℓ:

- אם ההחלטה **פתוחה** (לא נכתבה בפרומפט), g_ℓ היא ההסתברות שהמודל "ינחש" את מה שרציתם.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

נניח שהמשימה כוללת D החלטות, ונסמן אותן $\ell = 1, \dots, D$. נסמן ב-g_ℓ את ההסתברות שהתשובה מתאימה למפרט שלכם בהחלטה ℓ:

- אם ההחלטה **פתוחה** (לא נכתבה בפרומפט), g_ℓ היא ההסתברות שהמודל "ינחש" את מה שרציתם.
- אם ההחלטה **סגורה** (נכתבה במפורש), g_ℓ היא ההסתברות שהמודל **יציית** להוראה - גבוהה, אבל לא בהכרח 1.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

נניח שהמשימה כוללת D החלטות, ונסמן אותן $\ell = 1, \dots, D$. נסמן ב-g_ℓ את ההסתברות שהתשובה מתאימה למפרט שלכם בהחלטה ℓ:

- אם ההחלטה **פתוחה** (לא נכתבה בפרומפט), g_ℓ היא ההסתברות שהמודל "ינחש" את מה שרציתם.
- אם ההחלטה **סגורה** (נכתבה במפורש), g_ℓ היא ההסתברות שהמודל **יציית** להוראה - גבוהה, אבל לא בהכרח 1.

נניח שהתשובה תקינה **בדיוק** כשכל D ההחלטות מתאימות למפרט (כלומר אין מקור שגיאה נוסף), ושההחלטות בלתי-תלויות. אז:

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

נניח שהמשימה כוללת D החלטות, ונסמן אותן $\ell = 1, \dots, D$. נסמן ב-g_ℓ את ההסתברות שהתשובה מתאימה למפרט שלכם בהחלטה ℓ:

- אם ההחלטה **פתוחה** (לא נכתבה בפרומפט), g_ℓ היא ההסתברות שהמודל "ינחש" את מה שרציתם.
- אם ההחלטה **סגורה** (נכתבה במפורש), g_ℓ היא ההסתברות שהמודל **יציית** להוראה - גבוהה, אבל לא בהכרח 1.

נניח שהתשובה תקינה **בדיוק** כשכל D ההחלטות מתאימות למפרט (כלומר אין מקור שגיאה נוסף), ושההחלטות בלתי-תלויות. אז:

$$P_{\text{succ}}(x) = \prod_{\ell=1}^{D} g_\ell$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

$$P_{\text{succ}}(x) = \prod_{\ell=1}^{D} g_\ell$$

<u>מודל המכפלה:</u>

דוגמא: ארבע החלטות בפרומפט החלש של `is_prime`,
שכולן פתוחות ($D = 4$; ω מסמן את הקלט של הפונקציה):

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

$$P_{\text{succ}}(x) = \prod_{\ell=1}^{D} g_\ell$$

<u>מודל המכפלה:</u>

דוגמא: ארבע החלטות בפרומפט החלש של `is_prime`, שכולן פתוחות ($D = 4$; ω מסמן את הקלט של הפונקציה):

ℓ	החלטה	g_ℓ בפרומפט החלש (פתוחה)
1	שפה: Python	0.9
2	טיפול בקלט $\omega \leq 1$	0.5
3	יעילות (בדיקה עד $\sqrt{\omega}$)	0.4
4	פונקציה בלי `input()`	0.7

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

$$P_{\text{succ}}(x) = \prod_{\ell=1}^{D} g_\ell$$

<u>מודל המכפלה:</u>

דוגמא: ארבע החלטות בפרומפט החלש של `is_prime`, שכולן פתוחות ($D = 4$; ω מסמן את הקלט של הפונקציה):

ℓ	החלטה	g_ℓ בפרומפט החלש (פתוחה)
1	שפה: Python	0.9
2	טיפול בקלט $\omega \leq 1$	0.5
3	יעילות (בדיקה עד $\sqrt{\omega}$)	0.4
4	פונקציה בלי `input()`	0.7

$$P_{\text{succ}}(x_{\text{weak}}) = 0.9 \times 0.5 \times 0.4 \times 0.7 = 0.126$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

$$P_{\text{succ}}(x) = \prod_{\ell=1}^{D} g_\ell$$

<u>מודל המכפלה:</u>

דוגמא: ארבע החלטות בפרומפט החלש של `*is_prime*`, שכולן פתוחות ($D = 4$; ω מסמן את הקלט של הפונקציה):

ℓ	החלטה	g_ℓ בפרומפט החלש (פתוחה)
1	שפה: Python	0.9
2	טיפול בקלט $\omega \leq 1$	0.5
3	יעילות (בדיקה עד $\sqrt{\omega}$)	0.4
4	פונקציה בלי `input()`	0.7

$$P_{\text{succ}}(x_{\text{weak}}) = 0.9 \times 0.5 \times 0.4 \times 0.7 = 0.126$$

- כלומר, בערך פעם אחת מתוך שמונה התשובה תעמוד בכל הדרישות. הפרומפט החזק מחלק א' **סוגר את ארבע ההחלטות**: הוא מציין Python, את הטיפול ב-$\omega \leq 1$, את הבדיקה עד השורש, ומבקש פונקציה. אם המודל מציית לכל הוראה בהסתברות של $g_\ell = 0.95$ לכל ℓ, **אזי** :

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

$$P_{\text{succ}}(x) = \prod_{\ell=1}^{D} g_\ell$$

<u>מודל המכפלה:</u>

דוגמא: ארבע החלטות בפרומפט החלש של `is_prime`, שכולן פתוחות ($D = 4$; ω מסמן את הקלט של הפונקציה):

ℓ	החלטה	g_ℓ בפרומפט החלש (פתוחה)
1	שפה: Python	0.9
2	טיפול בקלט $\omega \leq 1$	0.5
3	יעילות (בדיקה עד $\sqrt{\omega}$)	0.4
4	פונקציה בלי `input()`	0.7

$$P_{\text{succ}}(x_{\text{weak}}) = 0.9 \times 0.5 \times 0.4 \times 0.7 = 0.126$$

- כלומר, בערך פעם אחת מתוך שמונה התשובה תעמוד בכל הדרישות. הפרומפט החזק מחלק א' **סוגר את ארבע ההחלטות**: הוא מציין Python, את הטיפול ב-$\omega \leq 1$, את הבדיקה עד השורש, ומבקש פונקציה. אם המודל מציית לכל הוראה בהסתברות של $g_\ell = 0.95$ לכל ℓ, **אזי** :

$$P_{\text{succ}}(x_{\text{strong}}) = 0.95^4 \approx 0.815$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

מסכנות:

1. **ההשפעה כפלית** ארבע החלטות "כמעט בסדר" ($g_\ell = 0.7$ כל אחת) נותנות $0.7^4 \approx 0.24$ - רוב התשובות ייכשלו.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

מסכנות:

1. **ההשפעה כפלית** ארבע החלטות "כמעט בסדר" ($g_\ell = 0.7$ כל אחת) נותנות $0.7^4 \approx 0.24$ - רוב התשובות ייכשלו.
2. **ההחלטה החלשה ביותר שולטת** סגירת החלטה ℓ **מחליפה גורם אחד במכפלה:** g_ℓ מוחלף ב- g'_ℓ (ההסתברות אחרי הסגירה), וכל שאר הגורמים נשארים כמו שהם.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

ניסוח פורמלי - פרומפט כהתניה, ואיכות כהסתברות:

<u>מודל המכפלה:</u>

מסכנות:

1. **ההשפעה כפלית** ארבע החלטות "כמעט בסדר" ($g_\ell = 0.7$ כל אחת) נותנות $0.7^4 \approx 0.24$ - רוב התשובות ייכשלו.
2. **ההחלטה החלשה ביותר שולטת** סגירת החלטה ℓ **מחליפה גורם אחד במכפלה:** g_ℓ מוחלף ב-g'_ℓ(ההסתברות אחרי הסגירה), וכל שאר הגורמים נשארים כמו שהם.

נניח עבור g_3 $0.4 \to 0.95$: $0.9 \times 0.5 \times 0.95 \times 0.7 \approx 0.299$ (לעומת 0.126).

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

עקרונות מעשיים:

לפני ששולחים פרומפט למשימה חשובה, רשמו לעצמכם 3-5 החלטות שאתם יודעים שחשובות לכם, ובדקו שכל אחת מהן כתובה בפרומפט.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

עקרונות מעשיים:

לפני ששולחים פרומפט למשימה חשובה, רשמו לעצמכם 3-5 החלטות שאתם יודעים שחשובות לכם, ובדקו שכל אחת מהן כתובה בפרומפט.

שאלה:

בפרומפט יש $D = 3$ החלטות פתוחות, עם $g_1 = 0.9,\ g_2 = 0.6,\ g_3 = 0.5$. מותר לכם להוסיף משפט אחד בלבד, שסוגר החלטה אחת (ומעלה את ה-g_ℓשלה ל-1). איזו החלטה תסגרו, ומה יהיה $P_{\text{succ}}(x)$ לפני ואחרי?

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ב' - למה ניסוח משנה: "החלטות פתוחות"

עקרונות מעשיים:

לפני ששולחים פרומפט למשימה חשובה, רשמו לעצמכם 3-5 החלטות שאתם יודעים שחשובות לכם, ובדקו שכל אחת מהן כתובה בפרומפט.

שאלה:
בפרומפט יש $D = 3$ החלטות פתוחות, עם $g_1 = 0.9,\ g_2 = 0.6,\ g_3 = 0.5$. מותר לכם להוסיף משפט אחד בלבד, שסוגר החלטה אחת (ומעלה את ה-g_ℓשלה ל-1). איזו החלטה תסגרו, ומה יהיה $P_{\text{succ}}(x)$ לפני ואחרי?

שאלה:
כמה החלטות פתוחות יש בפרומפט "*כתוב פונקציה שממיינת רשימה*"? מנו לפחות ארבע.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ג' - ארבע אבני היסוד

ארבע אבני היסוד הן דרך מסודרת **לסגור החלטות פתוחות**: **הקשר** · **מטרה** · **אילוצים** · **פורמט**. (לא כל פרומפט צריך את כל הארבע.)

אבן יסוד	מה היא סוגרת	דוגמה
1. הקשר (Context)	מה הפרויקט, מי ישתמש, טכנולוגיה וגרסה, כללים עסקיים	"אני בונה מערכת קופה לחנות בישראל, ב-Python. קודם הנחה, ואז מע"מ 18%; התוצאה מעוגלת לאגורות."
2. מטרה (Goal)	מה ייווצר ומה הוא יעשה — ודוגמת קלט ← פלט אחת לפחות	הפונקציה `remove_duplicates(items)` שומרת על סדר ההופעה הראשונה: `[3, 1, 3, 2, 1] → [3, 1, 2]`
3. אילוצים (Constraints)	טכניים (ספריות, גרסה), התנהגותיים (מקרי קצה), סגנון והיקף	"רק ספרייה סטנדרטית. רשימה ריקה ← `None`. ערך שאינו מספר ← `ValueError`."
4. פורמט (Format)	מה בדיוק מקבלים, ובאיזה מבנה	"רק הפונקציה, בלי קריאה ל-`input` ובלי `print`; אחריה 3 דוגמאות שימוש כהערות."

שלוש מסגרות פורמליות מאחורי אבני היסוד: המטרה היא **חוזה** (תנאי קדם ותנאי בתר); האילוצים נגזרים מ**מחלקות שקילות** של תחום הקלט Ω; הפורמט קובע את ה**חתימה** — שם הפונקציה, טיפוס הקלט וטיפוס הפלט.

Vibe Coding

שיעור 3 - חלק ג' - ארבע אבני היסוד

- ארבע אבני היסוד הן דרך מסודרת לסגור החלטות פתוחות: **הקשר** · **מטרה** · **אילוצים** · **פורמט**. (לא כל פרומפט צריך את כל הארבע).

3. **אילוצים (Constraints)** - מה שהמודל נוטה לפספס: מקרי קצה, ספריות שמותר או אסור להשתמש בהן, גרסאות, מגבלות גודל.

איך יודעים אילו מקרי קצה לכתוב כאילוצים? (גישת הנדסת תוכנה)

Vibe Coding

שיעור 3 - חלק ג' - ארבע אבני היסוד

- ארבע אבני היסוד הן דרך מסודרת לסגור החלטות פתוחות: **הקשר** · **מטרה** · **אילוצים** · **פורמט**. (לא כל פרומפט צריך את כל הארבע).

3. **אילוצים (Constraints)** - מה שהמודל נוטה לפספס: מקרי קצה, ספריות שמותר או אסור להשתמש בהן, גרסאות, מגבלות גודל.

איך יודעים אילו מקרי קצה לכתוב כאילוצים? (גישת הנדסת תוכנה)

a. **חלוקה למחלקות שקילות (equivalence partitioning)**. נסמן ב-Ω את **תחום הקלט** - קבוצת כל הקלטים האפשריים של הפונקציה - וב-ω קלט בודד ($\omega \in \Omega$). מחלקים את Ω לתת-קבוצות זרות $\Omega = \Omega_1 \cup \Omega_2 \cup \dots$ שמכסות אותו ($\Omega_1, \Omega_2, \dots$), כך שבתוך כל מחלקה הפונקציה אמורה "להתנהג אותו דבר". ההנחה: אם הקוד נכון על נציג אחד של מחלקה, סביר שהוא נכון על כולה.

Vibe Coding

שיעור 3 - חלק ג' - ארבע אבני היסוד

- ארבע אבני היסוד הן דרך מסודרת לסגור החלטות פתוחות: **הקשר** · **מטרה** · **אילוצים** · **פורמט**. (לא כל פרומפט צריך את כל הארבע).

3. **אילוצים (Constraints)** - מה שהמודל נוטה לפספס: מקרי קצה, ספריות שמותר או אסור להשתמש בהן, גרסאות, מגבלות גודל.

איך יודעים אילו מקרי קצה לכתוב כאילוצים? (גישת הנדסת תוכנה)

a. **חלוקה למחלקות שקילות (equivalence partitioning)**. נסמן ב-Ω את **תחום הקלט** - קבוצת כל הקלטים האפשריים של הפונקציה - וב-ω קלט בודד ($\omega \in \Omega$). מחלקים את Ω לתת-קבוצות זרות $\Omega = \Omega_1 \cup \Omega_2 \cup \dots$ שמכסות אותו ($\Omega_1, \Omega_2, \dots$), כך שבתוך כל מחלקה הפונקציה אמורה "להתנהג אותו דבר". ההנחה: אם הקוד נכון על נציג אחד של מחלקה, סביר שהוא נכון על כולה.

b. **ערכי גבול (boundary value analysis)**. באגים מתרכזים בגבולות שבין מחלקות. לכן בוחרים נציגים **על** הגבול **וליד** הגבול.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ג' - ארבע אבני היסוד

- ארבע אבני היסוד הן דרך מסודרת לסגור החלטות פתוחות: **הקשר** · **מטרה** · **אילוצים** · **פורמט**. (לא כל פרומפט צריך את כל הארבע).

3. **אילוצים (Constraints)** - מה שהמודל נוטה לפספס: מקרי קצה, ספריות שמותר או אסור להשתמש בהן, גרסאות, מגבלות גודל.

איך יודעים אילו מקרי קצה לכתוב כאילוצים? (גישת הנדסת תוכנה)

דוגמה: *is_prime*

מחלקה	תיאור	נציגים (כולל גבולות)	פלט נדרש
Ω_1	$\omega \leq 1$	1,0,-7	False
Ω_2	$\omega = 2$	2	True
Ω_3	זוגיים, $\omega > 2$	4,100	False
Ω_4	אי-זוגיים פריקים	$9 = 3^2$ (ריבוע - גבול של הלולאה עד $\sqrt{\omega}$), 25, $91 = 7 \times 13$ ، 561	False
Ω_5	ראשוניים אי-זוגיים עד מיליון	3,97,7919	True
Ω_6	ראשוניים מעל מיליון	1,000,000,007	True, פחות משניה

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ג' - ארבע אבני היסוד

- ארבע אבני היסוד הן דרך מסודרת לסגור החלטות פתוחות: **הקשר** · **מטרה** · **אילוצים** · **פורמט**. (לא כל פרומפט צריך את כל הארבע).

3. **אילוצים (Constraints)** - מה שהמודל נוטה לפספס: מקרי קצה, ספריות שמותר או אסור להשתמש בהן, גרסאות, מגבלות גודל.

איך יודעים אילו מקרי קצה לכתוב כאילוצים? (גישת הנדסת תוכנה)

דוגמה: *is_prime*

פלט נדרש	נציגים (כולל גבולות)	תיאור	מחלקה
False	1,0,-7	$\omega \leq 1$	Ω_1
True	2	$\omega = 2$	Ω_2
False	4,100	זוגיים, $\omega > 2$	Ω_3
False	$9 = 3^2$ (ריבוע - גבול של הלולאה עד $\sqrt{\omega}$), 25, $91 = 7 \times 13$ ، 561	אי-זוגיים פריקים	Ω_4
True	3,97,7919	ראשוניים אי-זוגיים עד מיליון	Ω_5
True, פחות משניה	1,000,000,007	ראשוניים מעל מיליון	Ω_6

כל מחלקה שאין לה כלל ברור בפרומפט היא החלטה פתוחה. כתיבת אילוץ לכל מחלקה = סגירת ההחלטות האלה.

Vibe Coding

שיעור 3 - חלק ד' - מרכיבים הכול

- ארבע אבני היסוד הן דרך מסודרת לסגור החלטות פתוחות: **הקשר** · **מטרה** · **אילוצים** · **פורמט**. (לא כל פרומפט צריך את כל הארבע).

תבנית פרומפט לשימוש חוזר:

הקשר: אני בונה ___ עבור ___, ב-___ (שפה + גרסה).
מטרה: כתוב ___ (שם + חתימה) ש___.
דוגמה: ___ → ___
אילוצים:
- ספריות: ___
- קלט ריק / לא תקין: ___
- ___
פורמט: ___ (רק קוד / קוד + הסבר / רק השינויים / + דוגמאות הרצה)

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ה' - מבנה ותוחמים: פרומפט ארוך שהמודל לא מתבלבל בו

ברגע שמדביקים לפרומפט **קוד**, **נתונים** או **הודעת שגיאה**, **הפרומפט מתערבב**: איפה נגמרות ההוראות שלכם ומתחיל החומר?

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ה' - מבנה ותוחמים: פרומפט ארוך שהמודל לא מתבלבל בו

ברגע שמדביקים לפרומפט **קוד**, **נתונים** או **הודעת שגיאה**, **הפרומפט מתערבב**: איפה נגמרות ההוראות שלכם ומתחיל החומר?

תוחמים (delimiters) הם סימנים שמפרידים בין חלקי הפרומפט:

1. **כותרות Markdown** - `### הקשר`, `### קוד`, `### מה אני רוצה`.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ה' - מבנה ותוחמים: פרומפט ארוך שהמודל לא מתבלבל בו

ברגע שמדביקים לפרומפט **קוד**, **נתונים** או **הודעת שגיאה**, **הפרומפט מתערבב**: איפה נגמרות ההוראות שלכם ומתחיל החומר?

תוחמים (delimiters) הם סימנים שמפרידים בין חלקי הפרומפט:

1. **כותרות Markdown** - `### הקשר`, `### קוד`, `### מה אני רוצה`.
2. **בלוקי קוד** - שלושה גרשיים הפוכים (```) לפני ואחרי הקוד.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ה' - מבנה ותוחמים: פרומפט ארוך שהמודל לא מתבלבל בו

ברגע שמדביקים לפרומפט **קוד**, **נתונים** או **הודעת שגיאה**, **הפרומפט מתערבב**: איפה נגמרות ההוראות שלכם ומתחיל החומר?

תוחמים (delimiters) הם סימנים שמפרידים בין חלקי הפרומפט:

1. **כותרות Markdown** - `### הקשר`, `### קוד`, `### מה אני רוצה`.
2. **בלוקי קוד** - שלושה גרשיים הפוכים (```) לפני ואחרי הקוד.
3. **תגיות בסגנון XML** - `<code>...</code>`, `<error>...</error>` הן גבולות ברורים שקל למודל לזהות.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ה' - מבנה ותוחמים: פרומפט ארוך שהמודל לא מתבלבל בו

ברגע שמדביקים לפרומפט **קוד**, **נתונים** או **הודעת שגיאה**, **הפרומפט מתערבב**: איפה נגמרות ההוראות שלכם ומתחיל החומר?

תוחמים (delimiters) הם סימנים שמפרידים בין חלקי הפרומפט:

1. **כותרות Markdown** - *### הקשר, ### קוד, ### מה אני רוצה.*
2. **בלוקי קוד** - שלושה גרשיים הפוכים (```) לפני ואחרי הקוד.
3. **תגיות בסגנון XML** - *<code>...</code>, <error>...</error>.* הן גבולות ברורים שקל למודל לזהות.

דוגמה:

```
### הקשר
פונקציה ממערכת הקופה (Python 3.11). מע"מ 18%.

### הקוד הנוכחי
```python
def total(prices):
    return sum(prices)  # TODO: להוסיף מע"מ
```

הבעיה
total([100, 50]) מחזיר 150. ציפיתי ל-177.0 (150 + מע"מ 18%).

מה אני רוצה
1. תקן כך שהתוצאה כוללת מע"מ, מעוגלת לאגורות.
2. אל תוסיף הנחה בשלב הזה.
החזר רק את הפונקציה המתוקנת והסבר של משפט אחד.
```

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

בכל סבב מוסיפים **בדיוק את מה שהתגלה כחסר**, כשלושה דברים:

1. **הודעת השגיאה המלאה** (כל ה-traceback, לא "יש שגיאה").
2. **הקלט שגרם לה**.
3. **מה ציפיתם שיקרה**.

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

בכל סבב מוסיפים **בדיוק את מה שהתגלה כחסר**, כשלושה דברים:

1. **הודעת השגיאה המלאה** (כל ה-traceback, לא "יש שגיאה").
2. **הקלט שגרם לה**.
3. **מה ציפיתם שיקרה**.

דוגמה: `return sum(grades) / len(grades)`. מריצים על הציונים שנקראו מקובץ - ["90", "85", "77"]

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

בכל סבב מוסיפים **בדיוק את מה שהתגלה כחסר**, כשלושה דברים:

1. **הודעת השגיאה המלאה** (כל ה-traceback, לא "יש שגיאה").
2. **הקלט שגרם לה**.
3. **מה ציפיתם שיקרה**.

דוגמה: `return sum(grades) / len(grades)`. מריצים על הציונים שנקראו מקובץ - ["90", "85", "77"]

```
Traceback (most recent call last):
  File "grades.py", line 4, in <module>
    average(["90", "85", "77"])
  File "grades.py", line 2, in average
    return sum(grades) / len(grades)
           ~~~^^^^^^^^
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

בכל סבב מוסיפים **בדיוק את מה שהתגלה כחסר**, כשלושה דברים:

1. **הודעת השגיאה המלאה** (כל ה-traceback, לא "יש שגיאה").
2. **הקלט שגרם לה**.
3. **מה ציפיתם שיקרה**.

דוגמה: `return sum(grades) / len(grades)`. מריצים על הציונים שנקראו מקובץ - ["90", "85", "77"]

הרצתי את הפונקציה על ["90", "85", "77"] (הציונים נקראים מקובץ טקסט, ולכן הם מחרוזות)
וקיבלתי:
TypeError: unsupported operand type(s) for +: 'int' and 'str"
ציפיתי ל-84.0. תקן כך שהפונקציה תקבל ציונים כמחרוזות. רשימה ריקה → None.

```
Traceback (most recent call last):
  File "grades.py", line 4, in <module>
    average(["90", "85", "77"])
  File "grades.py", line 2, in average
    return sum(grades) / len(grades)
           ~~~^^^^^^^^
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

נניח שאתם שולחים את **אותו** פרומפט x שוב ושוב (בשיחות חדשות), ושכל הרצה מצליחה בהסתברות $P_{\text{succ}}(x)$, באופן בלתי-תלוי. נסמן $P = P_{\text{succ}}(x)$ לקיצור, ובו-את G **מספר ההרצות עד ההצלחה הראשונה** (כולל ההרצה המוצלחת).
G מתפלג **גיאומטרית**:

$$\Pr[G = k] = (1-P)^{k-1}\,P, \qquad \mathbb{E}[G] = \tfrac{1}{P}, \qquad \Pr[G \le k] = 1-(1-P)^{k}$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

נניח שאתם שולחים את **אותו** פרומפט x שוב ושוב (בשיחות חדשות), ושכל הרצה מצליחה בהסתברות $P_{\text{succ}}(x)$, באופן בלתי-תלוי. נסמן $P = P_{\text{succ}}(x)$ לקיצור, ובו-G את **מספר ההרצות עד ההצלחה הראשונה** (כולל ההרצה המוצלחת).

G מתפלג **גיאומטרית**:

$$\Pr[G = k] = (1-P)^{k-1}\,P, \qquad \mathbb{E}[G] = \tfrac{1}{P}, \qquad \Pr[G \le k] = 1-(1-P)^{k}$$

- $\Pr[G = k]$: $k-1$ כישלונות (כל אחד בהסתברות $1-P$) ואחריהם הצלחה.
- $\mathbb{E}[G]$: מספר ההרצות **הממוצע** עד הצלחה.
- $\Pr[G \le k]$: הסיכוי להצליח בתוך k ניסיונות לכל היותר - המשלים של "כל k הניסיונות נכשלו", $(1-P)^{k}$.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

נניח שאתם שולחים את **אותו** פרומפט x שוב ושוב (בשיחות חדשות), ושכל הרצה מצליחה בהסתברות $P_{\text{succ}}(x)$, באופן בלתי-תלוי. נסמן $P = P_{\text{succ}}(x)$ לקיצור, ובו-G את **מספר ההרצות עד ההצלחה הראשונה** (כולל ההרצה המוצלחת).

G מתפלג **גיאומטרית**:

$$\Pr[G = k] = (1-P)^{k-1}\,P, \qquad \mathbb{E}[G] = \tfrac{1}{P}, \qquad \Pr[G \le k] = 1-(1-P)^{k}$$

- $\Pr[G = k]$: $k-1$ כישלונות (כל אחד בהסתברות $1-P$) ואחריהם הצלחה.
- $\mathbb{E}[G]$: מספר ההרצות **הממוצע** עד הצלחה.
- $\Pr[G \le k]$: הסיכוי להצליח בתוך k ניסיונות לכל היותר - המשלים של "כל k הניסיונות נכשלו", $(1-P)^{k}$.

שאלה: 1. פרומפט חלש P=0.126 ו-2. פרומפט חזק P=0.815 : מה התוחלת של G? מה ההסתברות להצלחה אחרי 3 ניסיונות?

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

נניח שאתם שולחים את **אותו** פרומפט x שוב ושוב (בשיחות חדשות), ושכל הרצה מצליחה בהסתברות $P_{\text{succ}}(x)$, באופן בלתי-תלוי. נסמן $P = P_{\text{succ}}(x)$ לקיצור, ובו-G את **מספר ההרצות עד ההצלחה הראשונה** (כולל ההרצה המוצלחת).

G מתפלג **גיאומטרית**:

$$\Pr[G = k] = (1-P)^{k-1}\,P, \qquad \mathbb{E}[G] = \tfrac{1}{P}, \qquad \Pr[G \le k] = 1-(1-P)^{k}$$

- $\Pr[G = k]$: $k-1$ כישלונות (כל אחד בהסתברות $1-P$) ואחריהם הצלחה.
- $\mathbb{E}[G]$: מספר ההרצות **הממוצע** עד הצלחה.
- $\Pr[G \le k]$: הסיכוי להצליח בתוך k ניסיונות לכל היותר - המשלים של "כל k הניסיונות נכשלו", $(1-P)^{k}$.

שאלה: 1. פרומפט חלש P=0.126 ו-2. פרומפט חזק P=0.815 : מה התוחלת של G? מה ההסתברות להצלחה אחרי 3 ניסיונות?

פרומפט	$P_{\text{succ}}(x)$	$\mathbb{E}[G]$	$[3 \ge \Pr[G$
חלש	0.126	7.94	0.332
חזק	0.815	1.23	0.994

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

<u>מודל עלות:</u>

נסמן ב-C_{write} את הזמן (בדקות) לכתוב את הפרומפט, וב-C_{check} את הזמן לבדוק תשובה אחת (קריאה + הרצה). הזמן הכולל עד קוד תקין, C, הוא זמן הכתיבה ועוד מספר ההרצות כפול זמן הבדיקה:

$$\mathbb{E}[C] = C_{\text{write}} + \mathbb{E}[G] \cdot C_{\text{check}} = C_{\text{write}} + \frac{C_{\text{check}}}{P_{\text{succ}}(x)}$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

<u>מודל עלות:</u>

נסמן ב-C_{write} את הזמן (בדקות) לכתוב את הפרומפט, וב-C_{check} את הזמן לבדוק תשובה אחת (קריאה + הרצה). הזמן הכולל עד קוד תקין, C, הוא זמן הכתיבה ועוד מספר ההרצות כפול זמן הבדיקה:

$$\mathbb{E}[C] = C_{\text{write}} + \mathbb{E}[G] \cdot C_{\text{check}} = C_{\text{write}} + \frac{C_{\text{check}}}{P_{\text{succ}}(x)}$$

עם $C_{\text{write}} = 0.5$ דקה לפרומפט החלש, 3 דקות לחזק, ו-$C_{\text{check}} = 3$ דקות:

$$\mathbb{E}[C_{\text{weak}}] = 0.5 + \frac{3}{0.126} \approx 24.3 \text{ min} \qquad \mathbb{E}[C_{\text{strong}}] = 3 + \frac{3}{0.815} \approx 6.7 \text{ min}$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

<u>מודל עלות:</u>

נסמן ב-C_{write} את הזמן (בדקות) לכתוב את הפרומפט, וב-C_{check} את הזמן לבדוק תשובה אחת (קריאה + הרצה). הזמן הכולל עד קוד תקין, C, הוא זמן הכתיבה ועוד מספר ההרצות כפול זמן הבדיקה:

$$\mathbb{E}[C] = C_{\text{write}} + \mathbb{E}[G] \cdot C_{\text{check}} = C_{\text{write}} + \frac{C_{\text{check}}}{P_{\text{succ}}(x)}$$

עם $C_{\text{write}} = 0.5$ דקה לפרומפט החלש, 3 דקות לחזק, ו-$C_{\text{check}} = 3$ דקות:

$$\mathbb{E}[C_{\text{weak}}] = 0.5 + \frac{3}{0.126} \approx 24.3 \text{ min} \qquad \mathbb{E}[C_{\text{strong}}] = 3 + \frac{3}{0.815} \approx 6.7 \text{ min}$$

מסקנה?:

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

<u>מודל עלות:</u>

נסמן ב-C_{write} את הזמן (בדקות) לכתוב את הפרומפט, וב-C_{check} את הזמן לבדוק תשובה אחת (קריאה + הרצה). הזמן הכולל עד קוד תקין, C, הוא זמן הכתיבה ועוד מספר ההרצות כפול זמן הבדיקה:

$$\mathbb{E}[C] = C_{\text{write}} + \mathbb{E}[G] \cdot C_{\text{check}} = C_{\text{write}} + \frac{C_{\text{check}}}{P_{\text{succ}}(x)}$$

עם $C_{\text{write}} = 0.5$ דקה לפרומפט החלש, 3 דקות לחזק, ו-$C_{\text{check}} = 3$ דקות:

$$\mathbb{E}[C_{\text{weak}}] = 0.5 + \frac{3}{0.126} \approx 24.3 \text{ min} \qquad \mathbb{E}[C_{\text{strong}}] = 3 + \frac{3}{0.815} \approx 6.7 \text{ min}$$

מסקנה?ההשקעה בכתיבת הפרומפט משתלמת כאשר זמן הבדיקה גדול.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ו' - איטרציה: לשפר בלולאה, ולהחזיר שגיאות בצורה נכונה

הלולאה **פרומפט** ← **תוצאה** ← **הרצה** ← **שיפור** היא **תהליך העבודה הרגיל**, ושניים-שלושה סבבים הם הנורמה, לא כישלון.

ניסוח פורמלי - כמה ניסיונות זה יעלה?

<u>מודל עלות:</u>

נסמן ב-C_{write} את הזמן (בדקות) לכתוב את הפרומפט, וב-C_{check} את הזמן לבדוק תשובה אחת (קריאה + הרצה). הזמן הכולל עד קוד תקין, C, הוא זמן הכתיבה ועוד מספר ההרצות כפול זמן הבדיקה:

$$\mathbb{E}[C] = C_{\text{write}} + \mathbb{E}[G] \cdot C_{\text{check}} = C_{\text{write}} + \frac{C_{\text{check}}}{P_{\text{succ}}(x)}$$

עם $C_{\text{write}} = 0.5$ דקה לפרומפט החלש, 3 דקות לחזק, ו-$C_{\text{check}} = 3$ דקות:

$$\mathbb{E}[C_{\text{weak}}] = 0.5 + \frac{3}{0.126} \approx 24.3 \text{ min} \qquad \mathbb{E}[C_{\text{strong}}] = 3 + \frac{3}{0.815} \approx 6.7 \text{ min}$$

מסקנה?:ההשקעה בכתיבת הפרומפט משתלמת כאשר זמן הבדיקה גדול.

הסתייגות: המודל מניח הרצות **זהות ובלתי-תלויות** של אותו פרומפט. **איטרציה היא שונה**: בכל סבב הפרומפט **משתנה**, ולכן גם P_{succ} משתנה - סבב תיקון טוב מעלה אותה. זה ההבדל בין "לנסות שוב ולקוות" (P_{succ} קבוע) לבין "לשפר את הפרומפט" (P_{succ} עולה).

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״תמונה שווה אלף מילים״

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot** = מצרפים לפרומפט כמה דוגמאות של קלט ← פלט, והמודל מזהה את הדפוס וממשיך אותו. המודל ממשיך דפוסים שהוא רואה בהקשר, ודוגמאות הן הדפוס הכי ברור שאפשר לתת. (Brown et al., 2020) GPT-3.

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot** = מצרפים לפרומפט כמה דוגמאות של קלט ← פלט, והמודל מזהה את הדפוס וממשיך אותו. המודל ממשיך דפוסים שהוא רואה בהקשר, ודוגמאות הן הדפוס הכי ברור שאפשר לתת. (Brown et al., 2020) GPT-3.
- **Zero-shot** - בלי דוגמאות.

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot** = מצרפים לפרומפט כמה דוגמאות של קלט ← פלט, והמודל מזהה את הדפוס וממשיך אותו. המודל ממשיך דפוסים שהוא רואה בהקשר, ודוגמאות הן הדפוס הכי ברור שאפשר לתת. (Brown et al., 2020) GPT-3.
- **Zero-shot** - בלי דוגמאות.
- **One-shot** - דוגמה אחת.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot** = מצרפים לפרומפט כמה דוגמאות של קלט ← פלט, והמודל מזהה את הדפוס וממשיך אותו. המודל ממשיך דפוסים שהוא רואה בהקשר, ודוגמאות הן הדפוס הכי ברור שאפשר לתת. (Brown et al., 2020) GPT-3.
- **Zero-shot** - בלי דוגמאות.
- **One-shot** - דוגמה אחת.

דוגמאות:

כתוב פונקציה ב-Python בשם normalize_phone(raw) שמנרמלת מספר טלפון ישראלי
לפורמט בינלאומי. דוגמאות:
"050-1234567" → "+972501234567"
"(03) 123-4567" → "+97231234567"
"+972 50 123 4567" → "+972501234567"
"0501234567" → "+972501234567"
"12345" → ValueError (מספר לא תקין)
רק ספרייה סטנדרטית.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot** = מצרפים לפרומפט כמה דוגמאות של קלט ← פלט, והמודל מזהה את הדפוס וממשיך אותו. המודל ממשיך דפוסים שהוא רואה בהקשר, ודוגמאות הן הדפוס הכי ברור שאפשר לתת. (Brown et al., 2020) GPT-3.
- **Zero-shot** - בלי דוגמאות.
- **One-shot** - דוגמה אחת.

דוגמאות:

כתוב פונקציה ב-Python בשם normalize_phone(raw) שמנרמלת מספר טלפון ישראלי
לפורמט בינלאומי. דוגמאות:
"050-1234567" → "+972501234567"
"(03) 123-4567" → "+97231234567"
"+972 50 123 4567" → "+972501234567"
"0501234567" → "+972501234567"
"12345" → ValueError (מספר לא תקין)
רק ספרייה סטנדרטית.

המר תיאור לשם קבוע בסגנון Python:
"user name" → USER_NAME
"max retry count" → MAX_RETRY_COUNT
"default time out" →

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot פורמלי:**

נסמן ב-$\mathcal{H}$ את **מרחב ההשערות**: כל הפונקציות שהמודל עשוי לכתוב בתגובה לבקשה. כל דוגמה בפרומפט היא זוג (קלט, פלט צפוי); נסמן את הדוגמה ה-u ב-$(\mathrm{in}_u, \mathrm{out}_u)$ - למשל ("050-1234567","+972501234567"). כל דוגמה פוסלת את כל הפונקציות שלא מקיימות אותה. אחרי u דוגמאות נשארות רק הפונקציות **העקביות** עם כולן:

$$\mathcal{H}_u = \{\, f \in \mathcal{H} \;:\; f(\mathrm{in}_1) = \mathrm{out}_1,\ \ldots,\ f(\mathrm{in}_u) = \mathrm{out}_u \,\}, \qquad \mathcal{H} = \mathcal{H}_0 \supseteq \mathcal{H}_1 \supseteq \mathcal{H}_2 \supseteq \ldots$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot פורמלי:**

נסמן ב-$\mathcal{H}$ את **מרחב ההשערות**: כל הפונקציות שהמודל עשוי לכתוב בתגובה לבקשה. כל דוגמה בפרומפט היא זוג (קלט, פלט צפוי); נסמן את הדוגמה ה-u ב-$(\mathrm{in}_u, \mathrm{out}_u)$ - למשל ("050-1234567","+972501234567"). כל דוגמה פוסלת את כל הפונקציות שלא מקיימות אותה. אחרי u דוגמאות נשארות רק הפונקציות **העקביות** עם כולן:

$$\mathcal{H}_u = \{\, f \in \mathcal{H} \;:\; f(\mathrm{in}_1) = \mathrm{out}_1,\ \ldots,\ f(\mathrm{in}_u) = \mathrm{out}_u \,\}, \qquad \mathcal{H} = \mathcal{H}_0 \supseteq \mathcal{H}_1 \supseteq \mathcal{H}_2 \supseteq \ldots$$

מסקנות:

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot פורמלי:**

נסמן ב-$\mathcal{H}$ את **מרחב ההשערות**: כל הפונקציות שהמודל עשוי לכתוב בתגובה לבקשה. כל דוגמה בפרומפט היא זוג (קלט, פלט צפוי); נסמן את הדוגמה ה-uב-$(\mathrm{in}_u, \mathrm{out}_u)$ - למשל ("050-1234567","+972501234567"). כל דוגמה פוסלת את כל הפונקציות שלא מקיימות אותה. אחרי u דוגמאות נשארות רק הפונקציות **העקביות** עם כולן:

$$\mathcal{H}_u = \{\, f \in \mathcal{H} \;:\; f(\mathrm{in}_1) = \mathrm{out}_1,\ \ldots,\ f(\mathrm{in}_u) = \mathrm{out}_u \,\}, \qquad \mathcal{H} = \mathcal{H}_0 \supseteq \mathcal{H}_1 \supseteq \mathcal{H}_2 \supseteq \ldots$$

מסקנות:

1. **דוגמה שלא פוסלת אף השערה - מבוזבזת.** (אין טעם לחזור על אותן הדוגמאות) **היא שייכת לאותה מחלקת שקילות.**

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot פורמלי:**

נסמן ב-$\mathcal{H}$ את **מרחב ההשערות**: כל הפונקציות שהמודל עשוי לכתוב בתגובה לבקשה. כל דוגמה בפרומפט היא זוג (קלט, פלט צפוי); נסמן את הדוגמה ה-u ב-$(\mathrm{in}_u, \mathrm{out}_u)$ - למשל ("050-1234567","+972501234567"). כל דוגמה פוסלת את כל הפונקציות שלא מקיימות אותה. אחרי u דוגמאות נשארות רק הפונקציות **העקביות** עם כולן:

$$\mathcal{H}_u = \{\, f \in \mathcal{H} \;:\; f(\mathrm{in}_1) = \mathrm{out}_1,\ \ldots,\ f(\mathrm{in}_u) = \mathrm{out}_u \,\}, \qquad \mathcal{H} = \mathcal{H}_0 \supseteq \mathcal{H}_1 \supseteq \mathcal{H}_2 \supseteq \ldots$$

מסקנות:

1. **דוגמה שלא פוסלת אף השערה - מבוזבזת.** (אין טעם לחזור על אותן הדוגמאות) **היא שייכת לאותה מחלקת שקילות.**
2. **בחירה טובה: נציג אחד מכל מחלקה -** מחלקות שקילות הן הכלי המשותף לאילוצים, ל-few-shot ולבדיקות.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ז' - Few-shot: ללמד דרך דוגמאות

״**תמונה שווה אלף מילים**״ ← ״**דוגמה שווה אלף הסברים**״ : פורמט פלט, סגנון שמות, כללי המרה. דוגמה אחת או שתיים עושות את העבודה של פסקת הסבר.

- **Few-shot פורמלי:**

נסמן ב-$\mathcal{H}$ את **מרחב ההשערות**: כל הפונקציות שהמודל עשוי לכתוב בתגובה לבקשה. כל דוגמה בפרומפט היא זוג (קלט, פלט צפוי); נסמן את הדוגמה ה-uב-$(\mathrm{in}_u, \mathrm{out}_u)$ - למשל ("050-1234567","+972501234567"). כל דוגמה פוסלת את כל הפונקציות שלא מקיימות אותה. אחרי u דוגמאות נשארות רק הפונקציות **העקביות** עם כולן:

$$\mathcal{H}_u = \{\, f \in \mathcal{H} \;:\; f(\mathrm{in}_1) = \mathrm{out}_1,\ \ldots,\ f(\mathrm{in}_u) = \mathrm{out}_u \,\}, \qquad \mathcal{H} = \mathcal{H}_0 \supseteq \mathcal{H}_1 \supseteq \mathcal{H}_2 \supseteq \ldots$$

מסקנות:

1. **דוגמה שלא פוסלת אף השערה - מבוזבזת.** (אין טעם לחזור על אותן הדוגמאות) **היא שייכת לאותה מחלקת שקילות.**
2. **בחירה טובה: נציג אחד מכל מחלקה -** מחלקות שקילות הן הכלי המשותף לאילוצים, ל-few-shot ולבדיקות.
3. **גם אחרי כל הדוגמאות, $\mathcal{H}_u$ לא מכילה פונקציה אחת בלבד.**

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

1. **פירוק משימה** ל-3-6 צעדים, שכל אחד מהם מייצר משהו שאפשר להריץ ולבדוק.

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

1. **פירוק משימה** ל-3-6 צעדים, שכל אחד מהם מייצר משהו שאפשר להריץ ולבדוק.
2. **צעד אחד** בכל פרומפט.

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

1. **פירוק משימה** ל-3-6 צעדים, שכל אחד מהם מייצר משהו שאפשר להריץ ולבדוק.
2. **צעד אחד** בכל פרומפט.
3. **הריצו ובדקו** את הצעד לפני שממשיכים.

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

1. **פירוק משימה** ל-3-6 צעדים, שכל אחד מהם מייצר משהו שאפשר להריץ ולבדוק.
2. **צעד אחד** בכל פרומפט.
3. **הריצו ובדקו** את הצעד לפני שממשיכים.
4. בצעד הבא - ציינו מה **כבר קיים** (או הדביקו אותו), כדי שהמודל יבנה עליו ולא מחדש.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

1. **פירוק משימה** ל-3-6 צעדים, שכל אחד מהם מייצר משהו שאפשר להריץ ולבדוק.
2. **צעד אחד** בכל פרומפט.
3. **הריצו ובדקו** את הצעד לפני שממשיכים.
4. בצעד הבא - ציינו מה **כבר קיים** (או הדביקו אותו), כדי שהמודל יבנה עליו ולא מחדש.

אני רוצה לבנות אפליקציית מטלות פשוטה ב-Python (שורת פקודה, בלי ממשק גרפי). אל תכתוב קוד עדיין. פרק את המשימה ל-4 עד 6 צעדים, כך שכל צעד מייצר פונקציה שאפשר להריץ ולבדוק לבד. לכל צעד: שם הפונקציה ומה היא עושה, במשפט אחד.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

1. **פירוק משימה** ל-3-6 צעדים, שכל אחד מהם מייצר משהו שאפשר להריץ ולבדוק.
2. **צעד אחד** בכל פרומפט.
3. **הריצו ובדקו** את הצעד לפני שממשיכים.
4. בצעד הבא - ציינו מה **כבר קיים** (או הדביקו אותו), כדי שהמודל יבנה עליו ולא מחדש.

אני רוצה לבנות אפליקציית מטלות פשוטה ב-Python (שורת פקודה, בלי ממשק גרפי). אל תכתוב קוד עדיין. פרק את המשימה ל-4 עד 6 צעדים, כך שכל צעד מייצר פונקציה שאפשר להריץ ולבדוק לבד. לכל צעד: שם הפונקציה ומה היא עושה, במשפט אחד.

1. make_task(title, due_date) - יוצרת מילון שמייצג מטלה אחת.
2. add_task(tasks, title, due_date) - מוסיפה מטלה חדשה לרשימה.
3. show_tasks(tasks) - מדפיסה את המטלות הפתוחות לפי תאריך יעד.
4. complete_task(tasks, title) - מסמנת מטלה כהושלמה.
5. main() - לולאת תפריט שמחברת את הכול.

ד"ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

<u>פורמלית - אמינות של שרשרת:</u>

משימה שמורכבת מ-M צעדים היא **מערכת טורית**: היא עובדת רק אם **כל** הצעדים עובדים. נסמן את הצעדים $s = 1, \dots, M$, וב-P_s את ההסתברות שצעד s ייכתב נכון. אם הצעדים בלתי-תלויים:

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

<u>פורמלית - אמינות של שרשרת:</u>

משימה שמורכבת מ-M צעדים היא **מערכת טורית**: היא עובדת רק אם **כל** הצעדים עובדים. נסמן את הצעדים $s = 1, \dots, M$, וב-P_s את ההסתברות שצעד s ייכתב נכון. בהנחה שהצעדים בלתי-תלויים:

$$\Pr[\text{all steps correct in one attempt}] = \prod_{s=1}^{M} P_s$$

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

<u>פורמלית - אמינות של שרשרת:</u>

משימה שמורכבת מ-M צעדים היא **מערכת טורית**: היא עובדת רק אם **כל** הצעדים עובדים. נסמן את הצעדים $s = 1, \dots, M$, וב-P_s את ההסתברות שצעד s ייכתב נכון. בהנחה שהצעדים בלתי-תלויים:

$$\Pr[\text{all steps correct in one attempt}] = \prod_{s=1}^{M} P_s$$

- **גישת הפרומפט היחיד - הכול בפרומפט אחד**. אם התשובה נכשלה, אנחנו לא יודעים איזה צעד שבור, ולכן מבקשים מחדש את **כל** M הצעדים. מספר הניסיונות מתפלג גיאומטרית עם הסתברות הצלחה $\prod_s P_s$, ולכן בממוצע $\frac{1}{\prod_s P_s}$ ניסיונות; כל ניסיון "עולה" M יחידות עבודה (כתיבה ובדיקה של M צעדים):

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

<u>פורמלית - אמינות של שרשרת:</u>

משימה שמורכבת מ-M צעדים היא **מערכת טורית**: היא עובדת רק אם **כל** הצעדים עובדים. נסמן את הצעדים $s = 1, \dots, M$, ובו-P_s את ההסתברות שצעד s ייכתב נכון. בהנחה שהצעדים בלתי-תלויים:

$$\Pr[\text{all steps correct in one attempt}] = \prod_{s=1}^{M} P_s$$

- **גישת הפרומפט היחיד - הכול בפרומפט אחד**. אם התשובה נכשלה, אנחנו לא יודעים איזה צעד שבור, ולכן מבקשים מחדש את **כל** הצעדים. מספר הניסיונות מתפלג גיאומטרית עם הסתברות הצלחה $\prod_s P_s$, ולכן בממוצע $\frac{1}{\prod_s P_s}$ ניסיונות; כל ניסיון "עולה" M יחידות עבודה (כתיבה ובדיקה של M צעדים):

$$\mathbb{E}[\text{cost}_{\text{single}}] = \frac{M}{\prod_{s=1}^{M} P_s}$$

ד"ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

<u>פורמלית - אמינות של שרשרת:</u>

משימה שמורכבת מ-M צעדים היא **מערכת טורית**: היא עובדת רק אם **כל** הצעדים עובדים. נסמן את הצעדים $s = 1, \dots, M$, ובּ-P_s את ההסתברות שצעד s ייכתב נכון. בהנחה שהצעדים בלתי-תלויים:

$$\Pr[\text{all steps correct in one attempt}] = \prod_{s=1}^{M} P_s$$

- **גישת הפרומפט היחיד - הכול בפרומפט אחד**. אם התשובה נכשלה, אנחנו לא יודעים איזה צעד שבור, ולכן מבקשים מחדש את **כל** הצעדים. מספר הניסיונות מתפלג גיאומטרית עם הסתברות הצלחה $\prod_s P_s$, ולכן בממוצע $\frac{1}{\prod_s P_s}$ ניסיונות; כל ניסיון "עולה" M יחידות עבודה (כתיבה ובדיקה של M צעדים):
$$\mathbb{E}[\text{cost}_{\text{single}}] = \frac{M}{\prod_{s=1}^{M} P_s}$$
- **גישת הפירוק - פירוק עם בדיקה אחרי כל צעד.** כל צעד נבדק לבד, ואם נכשל - חוזרים רק עליו. צעד s דורש בממוצע $1/P_s$ ניסיונות של יחידת עבודה אחת:

ד"ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.

<u>פורמלית - אמינות של שרשרת:</u>

משימה שמורכבת מ-M צעדים היא **מערכת טורית**: היא עובדת רק אם **כל** הצעדים עובדים. נסמן את הצעדים $s = 1, \dots, M$, ובּ-P_s את ההסתברות שצעד s ייכתב נכון. בהנחה שהצעדים בלתי-תלויים:

$$\Pr[\text{all steps correct in one attempt}] = \prod_{s=1}^{M} P_s$$

- **גישת הפרומפט היחיד - הכול בפרומפט אחד**. אם התשובה נכשלה, אנחנו לא יודעים איזה צעד שבור, ולכן מבקשים מחדש את **כל** הצעדים. מספר הניסיונות מתפלג גיאומטרית עם הסתברות הצלחה $\prod_s P_s$, ולכן בממוצע $\frac{1}{\prod_s P_s}$ ניסיונות; כל ניסיון "עולה" M יחידות עבודה (כתיבה ובדיקה של M צעדים):

$$\mathbb{E}[\text{cost}_{\text{single}}] = \frac{M}{\prod_{s=1}^{M} P_s}$$

- **גישת הפירוק - פירוק עם בדיקה אחרי כל צעד.** כל צעד נבדק לבד, ואם נכשל - חוזרים רק עליו. צעד s דורש בממוצע $1/P_s$ ניסיונות של יחידת עבודה אחת:

$$\mathbb{E}[\text{cost}_{\text{split}}] = \sum_{s=1}^{M} \frac{1}{P_s}$$

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.
<u>פורמלית - אמינות של שרשרת:</u>

דוגמה: מספרית (חושבה), $M = 5$ **צעדים עם אותה** P_s **:**

P_s	$\Pr[\text{all correct, one prompt}] = P_s^5$	$\mathbb{E}[\text{cost}_{\text{single}}]$	$\mathbb{E}[\text{cost}_{\text{split}}]$	on
0.95	0.774	6.46	5.26	1.2
0.90	0.590	8.47	5.56	1.5
0.80	0.328	15.26	6.25	2.4

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.
<u>פורמלית - אמינות של שרשרת:</u>

דוגמה: מספרית (חושבה), $M = 5$ **צעדים עם אותה** P_s **:**

P_s	$\Pr[\text{all correct, one prompt}] = P_s^5$	$\mathbb{E}[\text{cost}_{\text{single}}]$	$\mathbb{E}[\text{cost}_{\text{split}}]$	יחס
0.95	0.774	6.46	5.26	1.2
0.90	0.590	8.47	5.56	1.5
0.80	0.328	15.26	6.25	2.4

טעויות נפוצות:

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.
<u>פורמלית - אמינות של שרשרת:</u>

דוגמה: מספרית (חושבה), $M = 5$ **צעדים עם אותה** P_s **:**

P_s	$\Pr[\text{all correct, one prompt}] = P_s^5$	$\mathbb{E}[\text{cost}_{\text{single}}]$	$\mathbb{E}[\text{cost}_{\text{split}}]$	יחס
0.95	0.774	6.46	5.26	1.2
0.90	0.590	8.47	5.56	1.5
0.80	0.328	15.26	6.25	2.4

טעויות נפוצות:

1. **צעדים גדולים מדי** - "צעד 1: כל הלוגיקה; צעד 2: הממשק". זה לא פירוק.

ד"ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.
<u>פורמלית - אמינות של שרשרת:</u>

דוגמה: מספרית (חושבה), $M = 5$ **צעדים עם אותה** P_s **:**

P_s	$\Pr[\text{all correct, one prompt}] = P_s^5$	$\mathbb{E}[\text{cost}_{\text{single}}]$	$\mathbb{E}[\text{cost}_{\text{split}}]$	יחס
0.95	0.774	6.46	5.26	1.2
0.90	0.590	8.47	5.56	1.5
0.80	0.328	15.26	6.25	2.4

טעויות נפוצות:

1. **צעדים גדולים מדי** - "צעד 1: כל הלוגיקה; צעד 2: הממשק". זה לא פירוק.
2. **לא לבדוק בין הצעדים** - מבקשים את כל הצעדים ברצף ומריצים רק בסוף. כך חוזרים לבעיה המקורית.

Vibe Coding

שיעור 3 - חלק ח' - פירוק משימה גדולה לצעדים

המודל מייצר טוקן אחר טוקן ולא "מתכנן פרויקט", ולכן ככל שהתשובה ארוכה יותר - גדל הסיכוי לטעות.
נרצה: **משימה מפורקת = צעדים קטנים** שכל אחד נבדק לבד.
<u>פורמלית - אמינות של שרשרת:</u>

דוגמה: מספרית (חושבה), $M = 5$ **צעדים עם אותה** P_s **:**

P_s	$\Pr[\text{all correct, one prompt}] = P_s^5$	$\mathbb{E}[\text{cost}_{\text{single}}]$	$\mathbb{E}[\text{cost}_{\text{split}}]$	יחס
0.95	0.774	6.46	5.26	1.2
0.90	0.590	8.47	5.56	1.5
0.80	0.328	15.26	6.25	2.4

טעויות נפוצות:

1. **צעדים גדולים מדי** - "צעד 1: כל הלוגיקה; צעד 2: הממשק". זה לא פירוק.
2. **לא לבדוק בין הצעדים** - מבקשים את כל הצעדים ברצף ומריצים רק בסוף. כך חוזרים לבעיה המקורית.
3. **לא לציין מה כבר קיים** - המודל כותב מחדש את make_task בצעד 2, בגרסה קצת שונה, ועכשיו יש שתיים.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק ט' - שלוש טכניקות משלימות

טכניקה	מתי	דוגמה	מה היא **לא** עושה
1. שאלות הבהרה	לא בטוחים מה כל ההחלטות הפתוחות — תנו למודל למצוא אותן	"לפני שאתה כותב קוד — שאל אותי עד 6 שאלות על מה שאתה צריך לדעת כדי לכתוב אותה נכון. אל תכתוב קוד עד שאענה."	המודל לא תמיד ישאל את כל השאלות החשובות
2. תוכנית לפני קוד	לוגיקה לא טריוויאלית Chain of Thought (Wei et al., 2022)	"לפני הקוד, תאר את התוכנית ב-3-4 משפטים. אחר כך כתוב את הקוד."	לא מבטיחה שהקוד נכון — עדיין בודקים
3. תפקיד (Role)	לכוון סגנון וקהל יעד	"אתה מרצה שמסביר לסטודנטים בשנה ראשונה. הסבר מה עושה הקוד הבא, שורה אחר שורה."	לא מוסיף ידע: 162 תפקידים, 2,410 שאלות — אין שיפור עקבי בדיוק (Zheng et al., 2024)

איפה התפקיד "יושב": מודלי שיחה מקבלים רשימת הודעות, ולכל אחת תפקיד — `system` (הנחיית המערכת) או `user` (הודעת המשתמש). בגישה דרך API המבנה הזה גלוי. **כלל:** תפקיד מכוון סגנון; אבני היסוד מכוונות תוכן.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק י' - מדידה: איך יודעים שפרומפט אחד טוב יותר מאחר?

אמידה: כמה הרצות הצליחו?

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק י' - מדידה: איך יודעים שפרומפט אחד טוב יותר מאחר?

אמידה: כמה הרצות הצליחו?

מריצים פרומפט R פעמים (כל פעם בשיחה חדשה), ובודקים כל תשובה מול המפרט. אם c מתוכן עברו, האומד הטבעי של הסתברות ההצלחה הוא:

$$\hat{P}_{\text{succ}} = \frac{c}{R}$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק י' - מדידה: איך יודעים שפרומפט אחד טוב יותר מאחר?

אמידה: כמה הרצות הצליחו?

מריצים פרומפט R פעמים (כל פעם בשיחה חדשה), ובודקים כל תשובה מול המפרט. אם c מתוכן עברו, האומד הטבעי של הסתברות ההצלחה הוא:

$$\hat{P}_{\text{succ}} = \frac{c}{R}$$

מספר ההצלחות c מתפלג **בינומית**, $c \sim \text{Bin}(R, P_{\text{succ}})$: R ניסיונות בלתי-תלויים, שכל אחד מצליח בהסתברות P_{succ}.
עם R קטן האומד רועש מאוד, ולכן מדווחים גם **רווח סמך** (confidence interval) - טווח של ערכי P_{succ} מבוססי נתונים.

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק י' - מדידה: איך יודעים שפרומפט אחד טוב יותר מאחר?

אמידה: כמה הרצות הצליחו?

מריצים פרומפט R פעמים (כל פעם בשיחה חדשה), ובודקים כל תשובה מול המפרט. אם c מתוכן עברו, האומד הטבעי של הסתברות ההצלחה הוא:

$$\hat{P}_{\text{succ}} = \frac{c}{R}$$

מספר ההצלחות c מתפלג **בינומית**, $c \sim \text{Bin}(R, P_{\text{succ}})$: R ניסיונות בלתי-תלויים, שכל אחד מצליח בהסתברות P_{succ}.
עם R קטן האומד רועש מאוד, ולכן מדווחים גם **רווח סמך** (confidence interval) - טווח של ערכי P_{succ} מבוססי נתונים.

משתמשים ברווח וילסון (Wilson score interval) עם רמת סמך 95% ($z^* = 1.96$):

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק י' - מדידה: איך יודעים שפרומפט אחד טוב יותר מאחר?

אמידה: כמה הרצות הצליחו?

מריצים פרומפט R פעמים (כל פעם בשיחה חדשה), ובודקים כל תשובה מול המפרט. אם c מתוכן עברו, האומד הטבעי של הסתברות ההצלחה הוא:

$$\hat{P}_{\text{succ}} = \frac{c}{R}$$

מספר ההצלחות c מתפלג **בינומית**, $c \sim \text{Bin}(R, P_{\text{succ}})$: R ניסיונות בלתי-תלויים, שכל אחד מצליח בהסתברות P_{succ}.
עם R קטן האומד רועש מאוד, ולכן מדווחים גם **רווח סמך** (confidence interval) - טווח של ערכי P_{succ} מבוססי נתונים.

משתמשים ברווח וילסון (Wilson score interval) עם רמת סמך 95% ($z^* = 1.96$):

$$\frac{\hat{P} + \frac{(z^*)^2}{2R} \pm z^* \sqrt{\frac{\hat{P}(1-\hat{P})}{R} + \frac{(z^*)^2}{4R^2}}}{1 + \frac{(z^*)^2}{R}}$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק י' - מדידה: איך יודעים שפרומפט אחד טוב יותר מאחר?

אמידה: כמה הרצות הצליחו?

מריצים פרומפט R פעמים (כל פעם בשיחה חדשה), ובודקים כל תשובה מול המפרט. אם c מתוכן עברו, האומד הטבעי של הסתברות ההצלחה הוא:

$$\hat{P}_{\text{succ}} = \frac{c}{R}$$

מספר ההצלחות c מתפלג **בינומית**, $c \sim \text{Bin}(R, P_{\text{succ}})$: R ניסיונות בלתי-תלויים, שכל אחד מצליח בהסתברות P_{succ}.
עם R קטן האומד רועש מאוד, ולכן מדווחים גם **רווח סמך** (confidence interval) - טווח של ערכי P_{succ} מבוססי נתונים.

c/R	$\hat{P}_{\text{succ}}$	רווח וילסון 95%
0/5	0.00	[0.00, 0.43]
1/5	0.20	[0.04, 0.62]
4/5	0.80	[0.38, 0.96]
5/5	1.00	[0.57, 1.00]
9/10	0.90	[0.60, 0.98]
45/50	0.90	[0.79, 0.96]

משתמשים ברווח וילסון (Wilson score interval) עם רמת סמך 95% ($z^* = 1.96$):

כמה רחבים הרווחים בפועל? (חושב):

$$\frac{\hat{P} + \frac{(z^*)^2}{2R} \pm z^* \sqrt{\frac{\hat{P}(1-\hat{P})}{R} + \frac{(z^*)^2}{4R^2}}}{1 + \frac{(z^*)^2}{R}}$$

ד״ר ולד לנדא

Vibe Coding

שיעור 3 - חלק י' - מדידה: איך יודעים שפרומפט אחד טוב יותר מאחר?

אמידה: כמה הרצות הצליחו?

מריצים פרומפט R פעמים (כל פעם בשיחה חדשה), ובודקים כל תשובה מול המפרט. אם c מתוכן עברו, האומד הטבעי של הסתברות ההצלחה הוא:

$$\hat{P}_{\text{succ}} = \frac{c}{R}$$

מספר ההצלחות c מתפלג **בינומית**, $c \sim \text{Bin}(R, P_{\text{succ}})$: R ניסיונות בלתי-תלויים, שכל אחד מצליח בהסתברות P_{succ}.
עם R קטן האומד רועש מאוד, ולכן מדווחים גם **רווח סמך** (confidence interval) - טווח של ערכי P_{succ} מבוססי נתונים.

c/R	$\hat{P}_{\text{succ}}$	רווח וילסון 95%
0/5	0.00	[0.00, 0.43]
1/5	0.20	[0.04, 0.62]
4/5	0.80	[0.38, 0.96]
5/5	1.00	[0.57, 1.00]
9/10	0.90	[0.60, 0.98]
45/50	0.90	[0.79, 0.96]

משתמשים ברווח וילסון (Wilson score interval) עם רמת סמך 95% ($z^* = 1.96$):

$$\frac{\hat{P} + \frac{(z^*)^2}{2R} \pm z^* \sqrt{\frac{\hat{P}(1-\hat{P})}{R} + \frac{(z^*)^2}{4R^2}}}{1 + \frac{(z^*)^2}{R}}$$

כמה רחבים הרווחים בפועל? (חושב):

- **רווח סמך 95%: אם חוזרים על הניסוי פעמים רבות,**
  השיטה מייצרת רווח שמכיל את P_{succ} האמיתית בכ-95% מהמקרים.